



UTN- Tecnicatura Universitaria en Programación a Distancia
Programación II

TRABAJO PRÁCTICO INTEGRADOR

**FOOD STORE - SISTEMA DE GESTIÓN
DE PEDIDOS DE COMIDA**

INTEGRANTES:
KARINA D'ANGELO, CRISTIAN TAPIA,
EZEQUIEL GÓMEZ Y JAEI VOCOS.

**UTN – TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN A DISTANCIA – PROGRAMACIÓN 2**

INDICE

1. Introducción -----	3
2. Marco teórico -----	3
3. Descripción del proyecto -----	5
4. Arquitectura del sistema -----	6
5. Implementación -----	10
6. Capturas de funcionamiento -----	12
7. Dificultades y soluciones -----	15
8. Conclusiones -----	17
9. Bibliografía / Webgrafía -----	18
10. Enlace al repositorio y video explicativo -----	19

UTN – TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA – PROGRAMACIÓN 2

INTRODUCCIÓN

Objetivo del proyecto

El presente Trabajo Práctico Integrador consiste en el desarrollo de Food Store, un sistema de gestión de pedidos de comida implementado como una aplicación de consola en Java. El objetivo principal del sistema es permitir la administración de categorías, productos, usuarios y pedidos mediante operaciones de alta, baja, modificación y consulta (CRUD), facilitando la organización de la información relacionada con un negocio gastronómico.

Para su desarrollo se aplicaron los conceptos fundamentales de la Programación Orientada a Objetos (POO), tales como encapsulamiento, herencia, polimorfismo y abstracción. Asimismo, se utilizaron colecciones para el almacenamiento dinámico de datos en memoria, interfaces para definir comportamientos comunes entre las entidades y excepciones personalizadas para garantizar un manejo adecuado de errores y validaciones.

La estructura del proyecto fue organizada siguiendo una separación de responsabilidades entre las entidades del dominio, la lógica de negocio y la interfaz de usuario por consola. Esta organización permite obtener un código más ordenado, reutilizable y fácil de mantener, favoreciendo además la escalabilidad de la aplicación ante futuras mejoras.

A lo largo de este documento se describen las tecnologías empleadas, la arquitectura implementada, las decisiones técnicas adoptadas durante el desarrollo, las principales dificultades encontradas y las soluciones aplicadas. También se incluyen evidencias del funcionamiento del sistema mediante capturas de pantalla y una explicación general de las funcionalidades desarrolladas.

MARCO TEÓRICO

El presente proyecto fue desarrollado utilizando el lenguaje de programación Java, aplicando los principios fundamentales de la Programación Orientada a Objetos. Este paradigma permite organizar el código a partir de clases y objetos, representando entidades del mundo real dentro del sistema. En el caso de Food Store, las clases principales representan categorías, productos, usuarios, pedidos y detalles de pedido.

Uno de los conceptos centrales aplicados fue el encapsulamiento, que consiste en proteger los atributos internos de una clase y permitir el acceso a ellos mediante métodos públicos. De esta manera, cada entidad conserva el control sobre sus propios datos, favoreciendo un diseño más seguro, ordenado y fácil de mantener.

También se utilizó herencia a través de una clase abstracta llamada Base. Esta clase contiene atributos comunes a todas las entidades del sistema, como el identificador, el estado de eliminación lógica y la fecha de creación. Gracias a este mecanismo, las clases Categoría, Producto, Usuario, Pedido y DetallePedido pueden reutilizar una misma estructura común, evitando duplicación de código.

UTN – TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA – PROGRAMACIÓN 2

La abstracción se aplicó mediante la creación de clases que representan los elementos esenciales del sistema, dejando fuera detalles que no son necesarios para el funcionamiento principal. Además, se implementó una interfaz llamada Calculable, que define el comportamiento necesario para calcular el total de un pedido. Esta interfaz es implementada por la clase Pedido, permitiendo que el cálculo del total quede definido como una responsabilidad propia de dicha clase.

Otro elemento importante del proyecto fue el uso de colecciones. Como el sistema no utiliza una base de datos, la información se almacena en memoria durante la ejecución del programa mediante estructuras dinámicas como ArrayList. Estas colecciones permiten guardar, recorrer, buscar, modificar y listar objetos, haciendo posible el funcionamiento de las operaciones CRUD dentro de la aplicación de consola.

El sistema también incorpora enumeraciones, conocidas como enums, para representar conjuntos fijos de valores. En este proyecto se utilizaron enums para definir el rol de los usuarios, el estado de los pedidos y las formas de pago disponibles. Esto permite restringir las opciones posibles y evitar valores inválidos dentro del sistema.

Para lograr una aplicación más robusta, se implementó manejo de excepciones. Las excepciones permiten controlar situaciones de error sin que el programa finalice inesperadamente. En el proyecto se utilizaron excepciones personalizadas para validar casos como datos inválidos, entidades inexistentes o problemas relacionados con el stock de productos.

Además, se aplicó el concepto de baja lógica. En lugar de eliminar físicamente los objetos de las colecciones, las entidades poseen un atributo llamado eliminado. Cuando un registro se elimina, este atributo cambia a true, permitiendo conservar el historial de información y evitando inconsistencias en las relaciones entre los objetos.

Finalmente, el proyecto se organizó mediante una arquitectura por paquetes, separando las responsabilidades del sistema. Las entidades representan el modelo de dominio, los servicios contienen la lógica de negocio, las excepciones controlan errores específicos, los enums definen valores fijos, la interfaz establece un comportamiento común y el menú de consola permite la interacción con el usuario. Esta separación favorece la claridad del código, facilita su mantenimiento y permite que cada parte del sistema cumpla una función específica.

UTN – TECNICATURA UNIVERSITARIA

EN PROGRAMACIÓN A DISTANCIA – PROGRAMACIÓN 2

DESCRIPCIÓN DEL PROYECTO

Food Store es un sistema de gestión de pedidos de comida desarrollado como una aplicación de consola en Java. Su objetivo principal es permitir la administración de la información básica de un negocio gastronómico, organizando categorías, productos, usuarios y pedidos mediante operaciones de alta, baja, modificación y consulta.

El sistema permite trabajar con distintas entidades relacionadas entre sí. Las categorías sirven para clasificar los productos disponibles; los productos representan los artículos que pueden formar parte de un pedido; los usuarios son las personas asociadas a dichos pedidos; y los pedidos agrupan uno o más detalles, donde se indica qué producto fue seleccionado, en qué cantidad y cuál es el subtotal correspondiente.

La aplicación funciona a través de un menú principal de consola, desde el cual el usuario puede acceder a los distintos módulos del sistema. Cada módulo presenta opciones específicas para gestionar la información correspondiente, aplicando validaciones para evitar datos incorrectos y utilizando baja lógica para conservar el historial de los registros.

Funcionalidades principales

Gestión de Categorías

El módulo de categorías permite administrar las clasificaciones de los productos. Desde este apartado se pueden crear nuevas categorías, consultar el listado de categorías existentes, modificar sus datos y eliminarlas mediante baja lógica. Esto permite organizar el catálogo de productos de manera clara y mantener la información histórica dentro del sistema.

Gestión de Productos

El módulo de productos permite registrar los artículos disponibles para la venta. Cada producto se asocia a una categoría y cuenta con datos como nombre, descripción, precio, stock, imagen y estado de disponibilidad. El sistema permite crear productos, listarlos, modificar sus datos y eliminarlos lógicamente. Además, se aplican validaciones para impedir precios negativos, stock negativo o productos sin categoría válida.

Gestión de Usuarios

El módulo de usuarios permite administrar las personas registradas en el sistema. Cada usuario cuenta con datos identificatorios y de contacto, como nombre, apellido, mail, celular, contraseña y rol. El sistema permite crear, listar, modificar y eliminar usuarios mediante baja lógica. También se valida que el correo electrónico sea único, evitando la duplicación de usuarios con el mismo mail.

Gestión de Pedidos

El módulo de pedidos permite registrar compras realizadas por los usuarios. Para crear un pedido, se debe seleccionar un usuario existente y agregar uno o más detalles de pedido. Cada detalle contiene un producto, una cantidad y un subtotal calculado a partir del precio del producto y la cantidad solicitada.

UTN – TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA – PROGRAMACIÓN 2

Una vez cargados los detalles, el sistema calcula el total del pedido mediante la interfaz Calculable, implementada en la clase Pedido. También permite modificar el estado del pedido y la forma de pago, además de eliminar pedidos mediante baja lógica. De esta manera, se conserva la información histórica y se mantiene la coherencia de los datos relacionados.

En conjunto, estas funcionalidades permiten simular el funcionamiento básico de un sistema de gestión de pedidos de comida, aplicando conceptos fundamentales de Programación Orientada a Objetos, uso de colecciones, validaciones, excepciones personalizadas e interacción por consola.

UTN – TECNICATURA UNIVERSITARIA

EN PROGRAMACIÓN A DISTANCIA – PROGRAMACIÓN 2

ARQUITECTURA DEL SISTEMA

El proyecto fue organizado mediante una estructura de paquetes con separación de responsabilidades. Esta organización permite diferenciar las clases que representan el modelo de dominio, las clases encargadas de la lógica de negocio, las excepciones personalizadas, las enumeraciones, la interfaz utilizada para el cálculo de totales y la clase encargada de la interacción con el usuario por consola.

Estructura de paquetes

```
src/  
├── integrado.prog2  
│   ├── Main.java  
│   ├── entities  
│   ├── enums  
│   ├── exception  
│   ├── interfaces  
│   ├── service  
│   └── ui
```

La clase Main.java es el punto de entrada de la aplicación. Desde allí se inicia el menú principal del sistema.

El paquete entities contiene las clases principales del modelo, tales como Base, Categoría, Producto, Usuario, Pedido y DetallePedido.

El paquete enums contiene las enumeraciones utilizadas para representar valores fijos del sistema, como Rol, Estado y FormaPago.

El paquete exception contiene las excepciones personalizadas utilizadas para controlar errores específicos del sistema, como entidades inexistentes, datos inválidos o problemas relacionados con el stock.

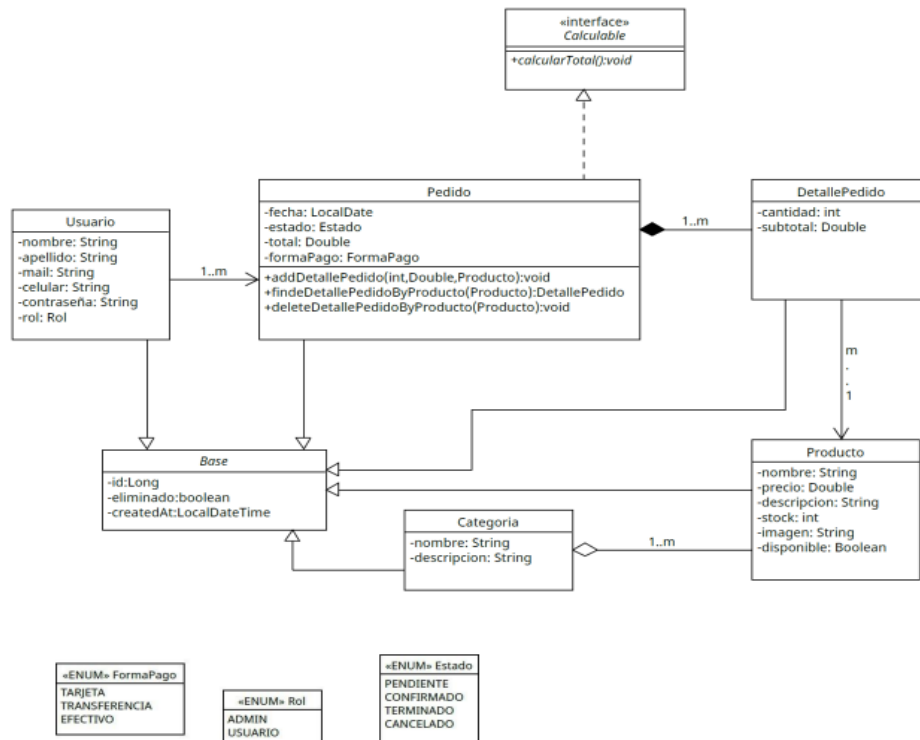
El paquete interfaces contiene la interfaz Calculable, que define el comportamiento necesario para calcular el total de un pedido.

El paquete service contiene las clases encargadas de la lógica de negocio y de la gestión de las colecciones en memoria. Allí se realizan operaciones como crear, listar, modificar, buscar y eliminar lógicamente las entidades.

El paquete ui contiene la clase AppMenu, encargada de la interacción con el usuario mediante consola.

UTN – TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA – PROGRAMACIÓN 2

UML



Entidades principales

Base

Es una clase abstracta que contiene atributos comunes a todas las entidades del sistema:

- id
- eliminado
- createdAt

Esta clase permite reutilizar una estructura común y aplicar herencia en las entidades principales.

Categoria

Representa las categorías utilizadas para organizar los productos del sistema. Una categoría puede estar asociada a varios productos.

Producto

Representa los productos disponibles dentro del sistema. Cada producto posee datos como nombre, descripción, precio, stock, imagen, disponibilidad y una categoría asociada.

Usuario

Representa a las personas registradas en el sistema que pueden estar asociadas a pedidos. Cada usuario posee datos como nombre, apellido, mail, celular, contraseña y rol.

Pedido

Representa una operación de compra realizada por un usuario. Contiene datos como

UTN – TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA – PROGRAMACIÓN 2

fecha, estado, forma de pago, total y una colección de detalles de pedido. Además, implementa la interfaz Calculable para calcular el total del pedido.

DetallePedido

Representa cada producto incluido dentro de un pedido. Contiene la cantidad solicitada, el subtotal correspondiente y la referencia al producto seleccionado.

En conjunto, esta arquitectura permite mantener el código organizado, separar responsabilidades y cumplir con el modelo orientado a objetos propuesto para el sistema Food Store.

UTN – TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA – PROGRAMACIÓN 2

IMPLEMENTACIÓN

Menú principal

El sistema inicia mostrando un menú principal por consola, desde el cual el usuario puede acceder a los distintos módulos de gestión de la aplicación. Las opciones disponibles permiten administrar categorías, productos, usuarios y pedidos.

Desde cada opción del menú principal se accede a un submenú específico con operaciones CRUD, es decir, alta, consulta, modificación y baja lógica de los registros correspondientes.

El menú principal permite acceder a:

- Gestión de categorías.
- Gestión de productos.
- Gestión de usuarios.
- Gestión de pedidos.

Esta organización facilita la navegación dentro del sistema y permite que cada funcionalidad se encuentre separada de manera clara.

Validaciones implementadas

Durante la carga y modificación de datos se implementaron distintas validaciones para evitar información incorrecta o inconsistente dentro del sistema.

Entre las principales validaciones se encuentran:

- El nombre no puede estar vacío.
- El precio de un producto debe ser mayor o igual a cero.
- El stock de un producto debe ser mayor o igual a cero.
- El mail de cada usuario debe ser único.
- La cantidad ingresada en un detalle de pedido debe ser mayor a cero.
- No se permite crear un producto asociado a una categoría inexistente o eliminada.
- No se permite crear un pedido sin un usuario válido.
- Se verifica la existencia de categorías, productos, usuarios y pedidos antes de modificarlos o eliminarlos.

Estas validaciones se complementan con el uso de excepciones personalizadas y bloques try-catch, evitando que el programa finalice inesperadamente ante un error de carga o una opción inválida.

Baja lógica

El sistema implementa baja lógica para las entidades principales. Esto significa que los registros no se eliminan físicamente de las colecciones, sino que se modifica el valor del atributo eliminado.

**UTN – TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN A DISTANCIA – PROGRAMACIÓN 2**

Cuando una entidad es dada de baja, se asigna:

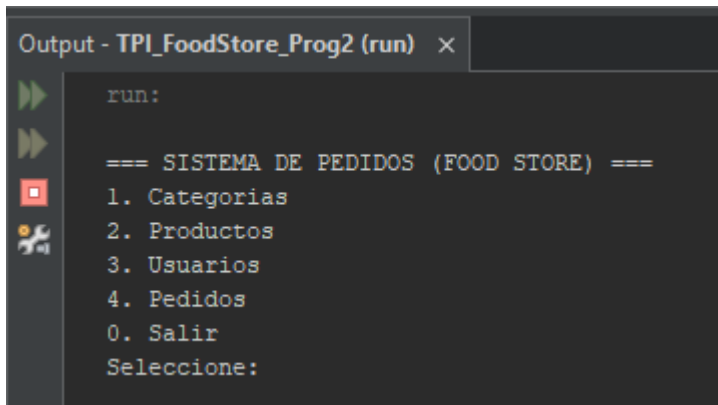
eliminado = true;

De esta manera, el objeto permanece en memoria, pero deja de mostrarse en los listados activos y no puede ser utilizado en nuevas operaciones cuando corresponde. Esta decisión permite conservar el historial de información y mantener la integridad de las relaciones entre las entidades del sistema.

UTN – TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA – PROGRAMACIÓN 2

CAPTURA FUNCIONAMIENTO

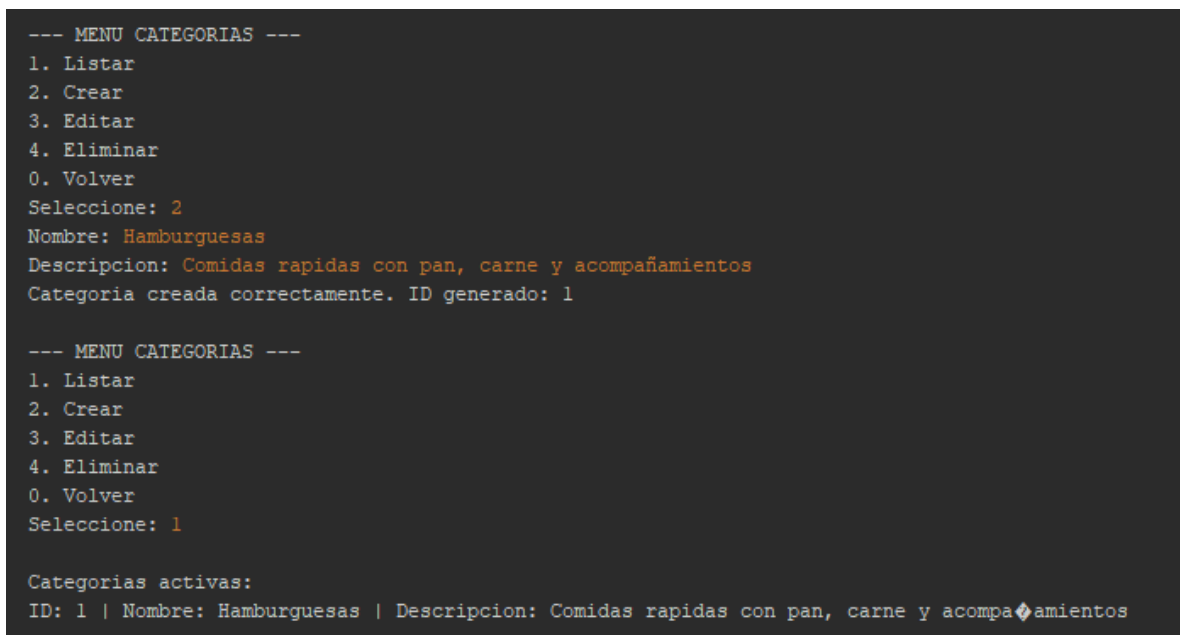
01 - Menu Principal



```
Output - TPI_FoodStore_Prog2 (run) x
run:

=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione:
```

02 - alta categoría y listado categoria



```
--- MENU CATEGORIAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Hamburguesas
Descripcion: Comidas rapidas con pan, carne y acompañamientos
Categoría creada correctamente. ID generado: 1

--- MENU CATEGORIAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1

Categorías activas:
ID: 1 | Nombre: Hamburguesas | Descripcion: Comidas rapidas con pan, carne y acompañamientos
```

UTN – TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA – PROGRAMACIÓN 2

03 – alta – producto y listado producto

```
Categorias activas:
ID: 1 | Nombre: Hamburguesas | Descripcion: Comidas rapidas con pan, carne y acompaamientos
Nombre: Hamburguesa simple
Descripcion: Pan, carne, queso y tomate
Precio: 5000
Stock: 5
Imagen: hamburguesa.jpg
Esta disponible? S/N: s
ID de categoria: 1
Producto creado correctamente. ID generado: 2

--- MENU PRODUCTOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1

Productos activos:
ID: 2 | Nombre: Hamburguesa simple | Precio: $5000.0 | Stock: 5 | Disponible: Si | Categoria: Hamburguesas
```

04 – alta usuario y listado usuarios

```
--- MENU USUARIOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Ana
Apellido: Zaccaro
Mail: anaz@gmail.com
Celular: 1123452345
Contrasena: 123456
Seleccione rol:
1. ADMIN
2. USUARIO
Opcion: 1
Usuario creado correctamente. ID generado: 3

--- MENU USUARIOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1

Usuarios activos:
ID: 3 | Nombre: Ana | Apellido: Zaccaro | Mail: anaz@gmail.com | Celular: 1123452345 | Rol: ADMIN
```

UTN – TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA – PROGRAMACIÓN 2

05 – creación pedido y total pedido

```
4. Pedidos
0. Salir
Seleccione: 4

--- MENU PEDIDOS ---
1. Listar
2. Crear pedido con detalles
3. Actualizar estado / forma de pago
4. Eliminar
0. Volver
Seleccione: 2

Usuarios activos:
ID: 3 | Nombre: Ana | Apellido: Zaccaro | Mail: anaz@gmail.com | Celular: 1123452345 | Rol: ADMIN
ID del usuario: 3
Seleccione forma de pago:
1. TARJETA
2. TRANSFERENCIA
3. EFECTIVO
Opcion: 3

Productos activos:
ID: 2 | Nombre: Hamburguesa simple | Precio: $5000.0 | Stock: 5 | Disponible: Si | Categoria: Hamburguesas
ID del producto: 2
Cantidad: 3
Detalle agregado correctamente.
Desea agregar otro producto al pedido? S/N: n
Pedido creado correctamente. ID generado: 4
Total del pedido: $15000.0
```

06 – modificación estado pedido

```
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 4

--- MENU PEDIDOS ---
1. Listar
2. Crear pedido con detalles
3. Actualizar estado / forma de pago
4. Eliminar
0. Volver
Seleccione: 3

Pedidos activos:
ID: 4 | Usuario: Ana Zaccaro | Fecha: 2026-06-17 | Estado: PENDIENTE | Forma de pago: EFECTIVO | Total: $15000.0
- ID: 5 | Producto: Hamburguesa simple | Cantidad: 3 | Subtotal: $15000.0
Ingrese ID del pedido a actualizar: 4
Desea cambiar el estado? S/N: s
Seleccione estado:
1. PENDIENTE
2. CONFIRMADO
3. TERMINADO
4. CANCELADO
Opcion: 4
Desea cambiar la forma de pago? S/N: n
Pedido actualizado correctamente.
```

UTN – TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA – PROGRAMACIÓN 2

07 – baja lógica

```
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 2

--- MENU PRODUCTOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 4

Productos activos:
ID: 2 | Nombre: Hamburguesa simple | Precio: $5000.0 | Stock: 2 | Disponible: Si | Categoría: Hamburguesas
Ingrese ID del producto a eliminar: 2
Confirma la baja logica? S/N: s
Producto eliminado logicamente.
```

08 – validación error

```
--- MENU PRODUCTOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2

Categorías activas:
ID: 1 | Nombre: Hamburguesas | Descripcion: Comidas rapidas con pan, carne y acompaamientos
Nombre: Pizza muzzarella
Descripcion: Pizza comun
Precio: 6000
Stock: -5
Imagen: pizza.png
Esta disponible? S/N: s
ID de categoria: 1
Error: El stock no puede ser negativo.
```

UTN – TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA – PROGRAMACIÓN 2

DIFICULTADES Y SOLUCIONES

Problema 1: Implementar la relación entre Pedido y DetallePedido

Una de las dificultades principales fue representar correctamente la relación entre un pedido y los productos incluidos en él. Un pedido puede contener varios detalles, y cada detalle debe indicar el producto seleccionado, la cantidad solicitada y el subtotal correspondiente.

Solución:

Se utilizó una colección de objetos DetallePedido dentro de la clase Pedido. De esta manera, cada pedido puede almacenar varios detalles asociados, permitiendo representar una relación uno a muchos entre ambas clases. Además, esta estructura permitió calcular el total del pedido a partir de la suma de los subtotales de cada detalle.

Problema 2: Controlar la duplicación de correos electrónicos

Otra dificultad fue evitar que se registraran usuarios con el mismo correo electrónico. Si el sistema permitía repetir mails, podían generarse datos duplicados o inconsistentes dentro de la colección de usuarios.

Solución:

Antes de crear un nuevo usuario, se recorrió la colección de usuarios existentes para verificar que no hubiera otro registro activo con el mismo mail. En caso de encontrar una coincidencia, el sistema muestra un mensaje de error y evita guardar el usuario duplicado.

Problema 3: Manejo de errores en la carga de datos

Durante la carga de datos por consola podían producirse errores, como ingresar valores inválidos, seleccionar identificadores inexistentes o intentar crear productos con stock negativo. Estos errores podían afectar el funcionamiento normal del programa.

Solución:

Se implementaron bloques try-catch y excepciones personalizadas para controlar los errores sin que el programa finalice inesperadamente. De esta manera, el sistema informa el problema mediante un mensaje claro y permite continuar utilizando el menú.

Problema 4: Implementar la baja lógica

Otra dificultad fue realizar eliminaciones sin borrar físicamente los objetos de las colecciones. Esto era necesario para conservar el historial de información y evitar inconsistencias entre entidades relacionadas.

Solución:

Se utilizó el atributo eliminado, heredado de la clase abstracta Base. Cuando una entidad se elimina, no se remueve de la colección, sino que se modifica su estado a eliminado = true. Luego, los listados muestran solamente las entidades activas, es decir, aquellas que no fueron eliminadas lógicamente.

UTN – TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA – PROGRAMACIÓN 2

CONCLUSIONES

El desarrollo del sistema Food Store permitió aplicar de manera integrada los principales conceptos trabajados en la materia Programación 2. A través de la construcción de una aplicación de consola en Java, fue posible implementar un sistema funcional basado en Programación Orientada a Objetos, utilizando clases, objetos, encapsulamiento, herencia, interfaces, colecciones y manejo de excepciones.

Durante el proyecto se trabajó con una estructura organizada por paquetes, separando las entidades del dominio, la lógica de negocio, las excepciones, los enums, la interfaz y la interacción con el usuario. Esta separación permitió obtener un código más claro, ordenado y fácil de mantener.

El uso de colecciones permitió almacenar la información en memoria durante la ejecución del programa, haciendo posible gestionar categorías, productos, usuarios y pedidos mediante operaciones CRUD. Además, se implementaron validaciones para evitar datos incorrectos, como precios negativos, stock inválido, usuarios inexistentes o correos electrónicos duplicados.

También se aplicó el concepto de baja lógica, lo cual permitió conservar el historial de los registros sin eliminarlos físicamente de las colecciones. Esto resulta importante para mantener la coherencia de la información y evitar inconsistencias entre entidades relacionadas.

En conclusión, el proyecto permitió integrar teoría y práctica en una aplicación concreta, fortaleciendo la comprensión de la Programación Orientada a Objetos y del diseño de sistemas organizados en capas. Food Store representa una solución básica pero funcional para la gestión de pedidos de comida, cumpliendo con los requisitos establecidos por la consigna del Trabajo Práctico Integrador.

UTN – TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA – PROGRAMACIÓN 2

BIBLIOGRAFÍA / WEBGRAFÍA

UTN - Tecnicatura Universitaria en Programación a Distancia. *Trabajo Práctico Integrador – Programación 2: Food Store – Sistema de Gestión de Pedidos de Comida.*

UTN - Tecnicatura Universitaria en Programación a Distancia. *Pautas de Corrección del Trabajo Práctico Integrador – Programación 2.*

UTN - Tecnicatura Universitaria en Programación a Distancia. *Programación II - Colecciones.*

UTN - Tecnicatura Universitaria en Programación a Distancia. *Complemento Práctico: El Fin de los Arreglos Estáticos.*

UTN - Tecnicatura Universitaria en Programación a Distancia. *Apunte: Enums en Java.*

UTN - Tecnicatura Universitaria en Programación a Distancia. *Apunte: Relaciones 1 a N en Java.*

UTN - Tecnicatura Universitaria en Programación a Distancia. *Representación UML de Relaciones 1 a N.*

UTN - Tecnicatura Universitaria en Programación a Distancia. *Complemento Práctico: Más allá de la Herencia Simple.*

UTN - Tecnicatura Universitaria en Programación a Distancia. *Complemento Práctico: Control de Daños y Robustez.*

UTN - Tecnicatura Universitaria en Programación a Distancia. *Complemento Práctico: Control Total: Lanzamiento y Recursos.*

**UTN – TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN A DISTANCIA – PROGRAMACIÓN 2**

ENLACES A REPOSITORIO Y VIDEO EXPLICATIVO

Repositorio: https://github.com/CTapia10/TPI_Programacion_Grupo-18

Video explicativo: <https://youtu.be/aevC0w2sbJc?si=yLF6XhvWtZV9pxLn>